MedVision AI

Medical Image AI Pipeline — Brain MRI Classification and Segmentation

Technical Foundations and Architecture

Doctum Consilium

Generated 2026-05-08

Abstract

Deep learning pipeline for brain MRI analysis using transfer learning, DVC data versioning, MLflow experiment tracking, and Docker deployment. This document covers the theoretical foundations, architectural decisions, and key technical concepts required to understand, contribute to, and maintain this repository.

Contents

Overview

MedVision AI implements a reproducible medical image analysis pipeline that:

1. Preprocesses and versions MRI datasets with DVC.
2. Trains classification and segmentation models (DenseNet121, ResNet50, Swin-V2) using PyTorch with transfer learning from ImageNet weights.
3. Tracks experiments, metrics, and artefacts with MLflow.
4. Serves inference via a FastAPI REST endpoint.

The pipeline targets brain tumour classification (glioma, meningioma, pituitary, no-tumour) and is designed for extensibility to segmentation tasks (BraTS-compatible datasets).

Architecture

```
Raw MRI dataset (NIfTI/DICOM)


DVC pipeline (preprocess → train → evaluate)


Versioned artefacts     MLflow Tracking
                       (metrics, params, models)

FastAPI inference endpoint


DICOM-SR / JSON prediction output
```

Key Technical Concepts

- Transfer Learning: fine-tuning ImageNet-pretrained CNNs on medical images; frozen backbone layers + trainable classification head; prevents overfitting on limited medical datasets.
- DenseNet121 architecture: dense connectivity $x_l = H_l([x_0, x_1, \ldots, x_{l-1}])$; feature reuse; proven on CheXNet (chest X-ray diagnosis).
- Data Augmentation: horizontal flip, rotation ±10ř, colour jitter; critical for medical image generalisation given dataset scarcity.
- Class Imbalance: weighted sampling, macro-F1 as primary metric rather than accuracy to avoid misleading results on imbalanced classes.
- Reproducibility: DVC pins dataset versions + pipeline stages; seeds fixed across NumPy/PyTorch/CUDA for deterministic training.
- MLflow: automatic logging of hyperparameters, per-epoch metrics, confusion matrices, and model artifacts for experiment comparison.

Data Flow

DICOM/NIfTI → DVC preprocess stage (resize, normalise) → PyTorch DataLoader (augmentation) → Transfer learning model → loss + metrics → MLflow log → checkpoint save → FastAPI inference (`POST /predict`).

Technology Stack

Component	Technology
Deep learning	PyTorch $\geq$ 2.0, torchvision
Model zoo	DenseNet121, ResNet50, Swin-V2-S
Data versioning	DVC 3.x
Experiment tracking	MLflow 2.x
Metrics	scikit-learn (F1, precision, recall, AUC)
Serving	FastAPI

Design Decisions

- DVC chosen over Git LFS for large dataset versioning with remote storage support.
- Macro-F1 as primary metric: clinically relevant — all classes matter equally.
- Three backbone options tested (DenseNet121, ResNet50, Swin-V2-S) to compare CNN vs. Vision Transformer performance on the same dataset.

Repository Structure

Listing 1: Top-level directory layout

```
medvision-ai/
  .github/
    copilot-instructions.md    # GitHub Copilot guardrails
    agents/
      ecr-secrets-agent.agent.md  # ECR secret rotation runbook
  CLAUDE.md                    # Claude Code guardrails
  GEMINI.md                    # Gemini CLI guardrails
  INFRASTRUCTURE.md            # Operational runbook
  ROADMAP.md                   # Execution log and roadmap
  README.md                    # Project overview
```

Contributing

1. Read `README.md`, `INFRASTRUCTURE.md`, and `ROADMAP.md` before any task.
2. Follow the Code Documentation Standard: every public function must have a docstring (Google-style for Python, JSDoc for TypeScript, XML for C#).
3. Run the guardrails validator before committing:

   ```
   git add -A && ./.guardrails/bin/validate_guardrails.sh
   ```

4. For deployment or destructive operations, always use a named tmux session:

   ```
   tmux new-session -A -s deploy-medvision-ai
   ```